

0.58. Plus large que haut : rectangle

`rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)` : un carré avec deux tailles, une pour X et une pour Y.

1. Ce qu'il faut comprendre

Un rectangle, c'est un carré qui n'a pas la même taille sur les deux axes. Au lieu d'une `taille`, il en a **deux** : la **largeur** (sur X) et la **hauteur** (sur Y).

Ce qui est nouveau ici : `rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)`, **huit** réglages : ceux de `carre`, avec `largeur` et `hauteur` à la place de `taille`.

- `largeur` et `hauteur` se comptent comme la taille d'un carré : la distance du centre au bord. La règle $2 \times n + 1$ vaut pour chacune : largeur 4 donne **9** cases de large, hauteur 2 donne **5** cases de haut, un rectangle de **9 × 5**.
- Avec la même largeur et la même hauteur, `rectangle` fait **un carré** : `rectangle(10, 8, A, 2, 2, ...)` fait le même tour que `carre(10, 8, A, 2, ...)`.

Le trajet : du centre, en diagonale, puis tout droit jusqu'au coin en haut à gauche ; le tour ; puis le même chemin à l'envers.

Les plus grandes tailles : largeur 9 (19 colonnes) et hauteur 8 (17 lignes).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <rectangle>` → [0.57.3. La fonction `rectangle\(\)` — tourner en rectangle](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1440 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
rectangle()    527 o ← main() – une fonction fournie par le langage
main()         42 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 4    52 o sur place, dans rectangle() + AdresseCase (10 o)
effacer()    x 3    45 o sur place, dans rectangle() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 4    12 o sur place, dans rectangle(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
rectangle() x 1    35 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <rectangle> // une tuile qui tourne en rectangle
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Un rectangle autour de la case (10, 8) :
    //
    // rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);
    //
    //      | | | | | | | | |
    //      | | | | | | | | | +-- tours : 1 tour
    //      | | | | | | | | | +----- vitesse : 250 ms par pas
    //      | | | | | | | | | +----- sens : 1, comme les aiguilles
    //      | | | | | | | | | +----- hauteur : 2 → 2 × 2 + 1 = 5 cases de haut
    //      | | | | | | | | | +----- largeur : 4 → 2 × 4 + 1 = 9 cases de large
    //      | | | | | | | | | +----- tuile : la lettre A
    //      +---+----- x, y : le centre (10, 8)
    //
    // colonne : 6 7 8 9 10 11 12 13 14
    // ligne 6 : → → → → → → → ↓ les côtés du haut et du bas :
    // ligne 7 : ↑ ↓ 8 pas (2 × largeur)
    // ligne 8 : ↑ A ↓
    // ligne 9 : ↑ ↓ les côtés de gauche et de droite :
    // ligne 10 : ↑ ← ← ← ← ← ← 4 pas (2 × hauteur)
    //
    // Le coin de départ est en haut à gauche, (6, 6) : le A y va du centre en
    // diagonale (2 pas), puis tout droit (2 pas), et en revient de même.
    rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);

    while (true) {
        image(); // le tour est fini : le A reste au centre
    }
}
```

3. Ce qui se passe, étape par étape

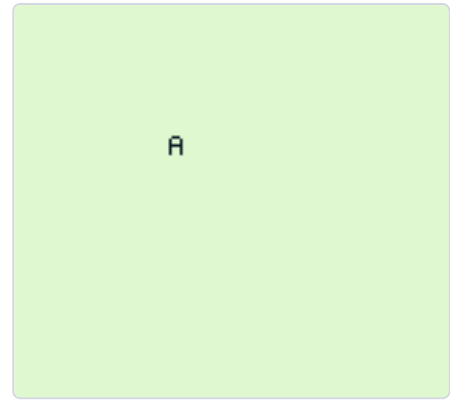
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



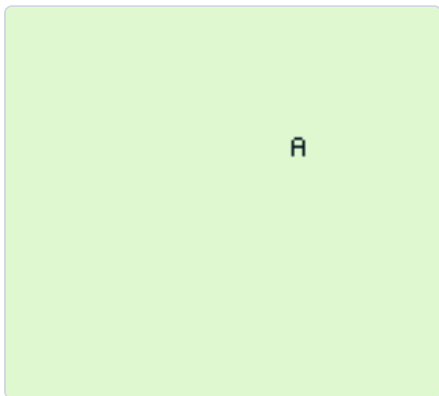
la toute première image — la console vient de démarrer — image 1



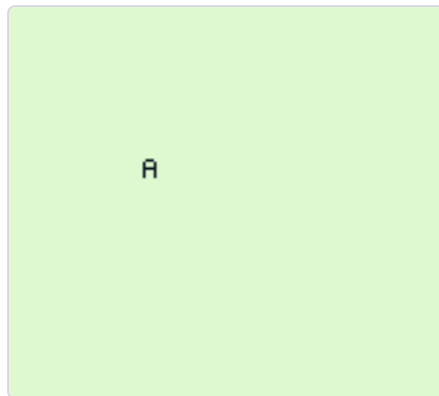
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait le tour d'un rectangle de 9 cases de large et 5 de haut autour de (10, 8), puis revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins : (6, 6), (14, 6), (14, 10), (6, 10)

✓ 9 de large, 5 de haut : jamais plus loin

✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.58.1. Plus large que haut : rectangle — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++